

GitHub Copilot

Essential Features



Inline Completions, Chat Window, Slash Commands, Chat Variables and Chat Participants

Copilot Chat: Overview

A conversational AI assistant integrated into your IDE



Dedicated Chat Panel

Sidebar or popup in your IDE with markdown-formatted responses



Natural Language

Type questions or requests — no special syntax required



Context Aware

References open files, selected code, or entire workspace



Multi-Turn Conversations

Maintains history for follow-up questions and iterative work



Code Actions

Insert code at cursor, create new files, or explain existing code



Slash Commands

Predefined actions: /explain, /fix, /tests, /doc, /optimize

When to Use Copilot Chat

5 categories of tasks where Chat excels

Understanding

- Explain unfamiliar code
- Learn new libraries
- Understand design patterns
- Clarify language concepts

Debugging

- Find bugs in code
- Fix errors with /fix
- Optimize slow queries
- Trace unexpected behavior

Architecture

- Explore framework options
- Design API structure
- Evaluate trade-offs
- Get recommendations

Refactoring

- Improve readability
- Extract reusable functions
- Modernize to async/await
- Add error handling

Docs & Tests

- Generate tests (/tests)
- Write JSDoc (/doc)
- Create usage examples
- Generate README

Inline Completion vs Copilot Chat

Inline Completions

- You know what you want to build
- You're implementing a clear spec
- Speed is critical
- The task is straightforward
- You need boilerplate or repetitive code
- You want to stay in flow stat

Chat Window

- You're exploring options
- You need to understand existing code
- You're debugging an issue
- You need explanations or learning
- The task is complex or ambiguous
- You want to iterate on solutions

Complementary, Not Competing

They're not alternatives — they're tools for different phases of development

 **Inline** = Your Speed Tool

 **Chat** = Your Thinking Tool

A Typical Workflow

1 Chat: "Explain how this authentication middleware works"

Understanding

2 Inline: Write a new route using that middleware

Execution

3 Chat: "Why is this route returning 401?"

Debugging

4 Inline: Fix the bug based on Chat's explanation

Execution

Expert developers switch between them fluidly based on their current need

Inline Completions vs Copilot Chat

Aspect	Inline Completions	Copilot Chat
Trigger	Automatic as you type	Manual — open panel, ask question
Display	Gray ghost text in editor	Dedicated chat panel with markdown
Interaction	Accept (Tab), Reject (Esc), Cycle (Alt+)]	Multi-turn conversation, follow-ups
Response Time	200–500ms (fast)	1–5 seconds (more thorough)
Output Scope	1–20 lines of code	Code + explanations, unlimited
Context	Current file + open tabs	File + selection + conversation history
Best For	Writing code quickly (execution)	Learning, debugging (thinking)

Inline: Triggers & Shortcuts

Automatic Triggers

- New line after comment → suggestion appears
- Function signature → body suggested
- Opening brace { → content suggested
- Typing pause (200–300ms) → triggers suggestion
- After keyword (if, for, return) → completion follows
- Incomplete statement (const users =) → value

Manual Trigger: **Alt + ** (macOS: Option + \)

Keyboard Shortcuts

Action	Shortcut
Accept Entire Suggestion	Tab
Accept Word-by-Word	Ctrl + →
Reject Suggestion	Esc
Next Alternative	Alt +]
Previous Alternative	Alt + [
Trigger Manually	Alt + \

Slash Commands

Predefined actions for common tasks — type / in chat

Command	Description	Example
/clear	Start a new chat session	/clear (no prompt needed — resets the chat context)
/explain	Explain how the code in your active editor works	/explain how does the middleware pipeline work in this file?
/fix	Propose a fix for problems in the selected code	/fix the null reference exception in the selected method
/fixTestFailure	Find and fix a failing test	/fixTestFailure why is the TodoController POST test failing?
/help	Quick reference and basics of using GitHub Copilot	/help (no prompt needed — shows available commands and tips)
/new	Create a new project	/new create a .NET Web API project with Entity Framework and SQL Server
/tests	Generate unit tests for the selected code	/tests generate xUnit tests for the selected TodoController actions

Context Variables

Precise control with # — type # in chat to reference specific context

Variable	What It References	Example
#block	Includes the current block of code in the prompt	explain what this #block of code does
#class	Includes the current class in the prompt	review #class for any SOLID principle violations
#comment	Includes the current comment in the prompt	write code that implements the logic described in #comment
#file	Includes the current file's content in the prompt	summarize what #file does and suggest improvements
#function	Includes the current function or method in the prompt	/tests generate unit tests for #function
#line	Includes the current line of code in the prompt	explain what #line does and why it might throw an exception
#path	Includes the file path in the prompt	based on #path suggest the correct namespace for this file
#project	Includes the project context in the prompt	suggest a folder structure improvement based on #project
#selection	Includes the currently selected text in the prompt	/explain #selection and suggest a more readable version

Power User Combinations

Combine slash commands + context variables for maximum effect

Example 1: Scoped Testing & Optimization

```
/tests #selection
```

```
/optimize #selection for better  
performance without changing behavior
```

Example 2: Multi-File References

Compare #file:oldAuth.js with
#file:newAuth.js and explain differences

Extract shared logic from #file:auth.js
and #file:validation.js into utilities

Example 3: Debug with Terminal Context

The test in #selection is failing with
#terminalSelection error.
What's the root cause and how do I fix it?

Example 4: Document with Type References

```
/doc for all functions in #file
```

```
/doc #selection with usage examples  
that reference #file:types.ts interfaces
```

Chat Participants

Specialized agents that provide Copilot with specific context

@workspace

- Architecture: "Where is authentication handled?"
- Docs: "Create a README for this project"
- Dependencies: "What external APIs do we use?"
- Refactoring: "What files would I need to change to add caching?"

@terminal

- Errors: "Why did my build fail?"
- Commands: "How to run tests in watch mode?"
- Output: "What does this error mean?"
- Scripts: "Create a deployment script"

@vscode

- Settings: "How to change font size?"
- Extensions: "Find a linter for Python"
- Keybindings: "Shortcut for multi-cursor"
- Themes: "Switch to dark mode"

@github

- Search repos, issues, and PRs
- Code search across organization
- Copilot knowledge bases (Enterprise)
- MCP skills on github.com

Next Edit Suggestions (NES)

Predicts both the location and content of your next code edit

How NES Works

- 1 Analyzes your recent editing history
- 2 Predicts what you'll want to change next
- 3 Shows arrow in editor gutter at predicted location
- 4 Press Tab to navigate to the suggestion
- 5 Press Tab again to accept the edit
- 6 Works across the entire file — not just at cursor

Best For

- > **Refactoring:** Renaming variables across multiple locations
- > **Data Structures:** Adding fields and updating all references
- > **Code + Comments:** Suggests changes to code, comments, and tests
- > **Bug Fixes:** Identifies related fixes after your initial change
- > **Missing Imports:** Detects and adds imports after new code

Unlike traditional ghost text (cursor only), NES predicts edits anywhere in your file

Thank You!

Questions & Discussion